```
ls | more
```

در این حالت، فرمان ls به عنوان یک تولیدکننده بوده و خروجی آن به مصرف فرمان more می‌رسد.

سیستم‌های ویندوز نیز در پوسته‌ی DOS، یک فرمان more با کارکرد مشابه همتای یونیکس خود دارند. (سیستم‌های یونیکس یک فرمان more هم دارند، اما در سبک جالب و غیرجدی[1] که در یونیکس مرسوم است، فرمان less به‌واقع کارکرد بیشتری از more فراهم می‌کند!) پوسته‌ی DOS هم برای برپایی یک لوله از کاراکتر | استفاده می‌نماید. فقط تفاوت در گرفتن فهرست دایرکتوری است که DOS به جای فرمان ls از dir، آن‌گونه که در زیر نشان داده شده، استفاده می‌کند:

```
dir | more
```

۳-۸ ارتباط در سیستم‌های کارفرما-کارگزار

در بخش ۳-۴، چگونگی ارتباط پردازه‌ها را با استفاده از حافظه‌ی مشترک و عبور پیغام توصیف کردیم. این تکنیک‌ها می‌توانند جهت ارتباط سیستم‌های کارفرما-کارگزار (بخش ۱-۱۰-۳) نیز استفاده شوند. در این بخش، دو راهبرد دیگر را برای ارتباط سیستم‌های کارفرما-کارگزار مطرح می‌کنیم که عبارتند از: سوکت‌ها[2] و فراخوانی‌های روال دور[3] (RPC). همان‌گونه که در پوشش ما از RPC خواهیم دید، آن‌ها نه تنها برای محاسبات کارفرما-کارگزار مفید هستند، بلکه اندروید هم از روال‌های دور به‌صورت شکلی از IPC میان پردازه‌های در حال اجرا بر روی سیستم یکسان استفاده می‌کند.

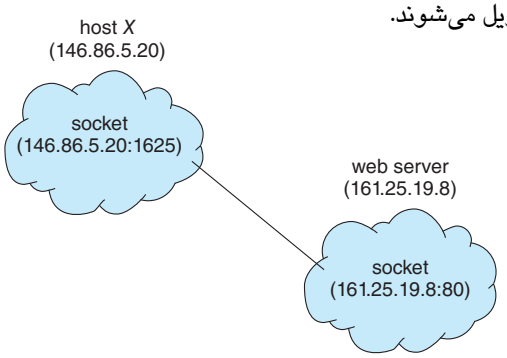
۳-۸-۱ سوکت‌ها

سوکت به عنوان نقطه‌ی انتهایی ارتباط تعریف می‌شود. یک جفت پردازه که در یک شبکه با هم در ارتباطند یک جفت سوکت—یکی برای هر پردازه—را به‌کار می‌برند. هر سوکت با یک آدرس IP همراه با یک شماره درگاه الحاقی به آن مشخص می‌شود. سوکت‌ها به طور کلی از یک معماری کارفرما-کارگزار استفاده می‌نمایند. کارگزار با گوش‌ایستادن به یک درگاه مشخص به انتظار درخواست‌های سر راه می‌نشیند. همین که درخواستی دریافت می‌شود، کارگزار، اتصالی را از سوکت کارفرما برای تکمیل اتصال می‌پذیرد. کارگزارهایی که سرویس‌های خاصی (همچون SSH، FTP و HTTP) را پیاده‌سازی می‌کنند به درگاه‌های شناخته‌شده‌ای گوش می‌دهند (کارگزار SSH به پورت ۲۲؛ و کارگزار FTP به پورت ۲۱؛ و کارگزار وب یا HTTP به پورت ۸۰ گوش می‌دهد). تمامی درگاه‌های زیر ۱۰۲۴ *آشنا* تلقی می‌شوند و برای پیاده‌سازی سرویس‌های معمول استفاده می‌شوند.

زمانی که پردازه‌ی کارفرمایی درخواستی برای ایجاد یک اتصال صادر می‌کند، توسط کامپیوتر میزبان خود به یک درگاه گمارده می‌شود. این درگاه، یک شماره‌ی دلخواه بزرگ‌تر از ۱۰۲۴ دارد. برای مثال، اگر کارفرمایی از میزبان X به آدرس IP، 146.85.5.20 بخواهد اتصالی به یک کارگزار وب (که به درگاه ۸۰ گوش می‌دهد) با آدرس 161.25.19.8 برپا کند، میزبان X می‌تواند

1. Tongue-in-cheek
2. Sockets
3. Remote procedure calls

به درگاه ۱۶۲۵ گمارده شود. اتصال شامل یک زوج سوکت خواهد بود: (146.86.5.20:1625) بر روی میزبان X و (161.25.19.8:80) بر روی کارگزار وب. این وضعیت در شکل ۳-۲۶ نشان داده می‌شود. بسته‌هایی که مابین میزبان‌ها در گردش هستند، مطابق شماره‌ی درگاه مقصدشان به پردازه‌ی مناسبی تحویل می‌شوند.



شکل ۳-۲۶ ارتباط با استفاده از سوکت‌ها.

تمامی اتصال‌ها بایستی یکتا باشند. بنابراین، اگر پردازه‌ی دیگری از میزبان X بخواهد اتصال دیگری به همان کارگزار وب برپا کند، بایستی به یک شماره درگاه بزرگ‌تر از ۱۰۲۴ و به غیر از ۱۶۲۵ گمارده شود. این کار تضمین می‌کند که زوج‌سوکت تمامی اتصال‌ها یکتا خواهند بود.

هرچند که اکثر مثال‌های این کتاب به زبان C هستند، برای فهم سوکت‌ها از کد جاوا استفاده می‌کنیم که رابط بسیار ساده‌تری برای سوکت‌ها فراهم کرده و برای برنامه‌های سودمند شبکه نیز از کتابخانه‌ای غنی برخوردار است.

جاوا سه نوع سوکت متفاوت فراهم می‌کند. **سوکت‌های اتصال‌گرا**[1] **(TCP)** که با کلاس Socket پیاده‌سازی می‌شوند. **سوکت‌های بی‌اتصال**[2] **(UDP)** که از کلاس DatagramSocket استفاده می‌کنند. بالاخره، کلاس MulticastSocket که زیرکلاسی از کلاس DatagramSocket می‌باشد. سوکت چندپخشی[3] امکان ارسال داده‌ها را به گیرندگان متعدد ممکن می‌سازد.

مثال ما، یک کارگزار تاریخ را توصیف می‌کند که از سوکت‌های اتصال‌گرای TCP استفاده می‌کند. این عملیات به کارفرماها امکان می‌دهد تا تاریخ و ساعت فعلی را از کارگزار درخواست نمایند. کارگزار به درگاه ۶۰۱۳ گوش می‌دهد، اگرچه این شماره درگاه می‌تواند هر شماره‌ی دلخواه استفاده نشده، بزرگ‌تر از ۱۰۲۴ هم باشد. زمانی که اتصالی دریافت می‌شود، کارگزار تاریخ و ساعت را به کارفرما بر می‌گرداند.

کارگزار تاریخ در شکل ۳-۲۷ نشان داده می‌شود. کارگزار، یک ServerSocket ایجاد می‌کند که مشخص می‌کند به درگاه ۶۰۱۳ گوش خواهد داد. در ادامه، کارگزار با فراخوانی متد accept() در درگاه گوش می‌ایستد. کارگزار در متد accept() به انتظار درخواست اتصالی از یک کارفرما

1. Connection-oriented
2. Connectionless
3. Multicast

مسدود می‌ماند. زمانی که درخواست اتصالی دریافت می‌شود، تابع accept() یک سوکت بر می‌گرداند که کارگزار می‌تواند برای ارتباط با کارفرما استفاده کند.

```
import java.net.*;
import java.io.*;
public class DateServer
{
        public static void main(String[] args) {
        try {
                ServerSocket sock = new ServerSocket(6013);

                /* now listen for connections */
                while (true) {
                        Socket client = sock.accept();
                        PrintWriter pout = new
                        PrintWriter(client.getOutputStream(),true);

                        /* write the Date to the socket */
                        pout.println(new java.util.Date().toString());

                        /* close the socket and resume */
                        /* listening for connections */
                        client.close();
                }
        }
        catch (IOException ioe) {
                        System.err.println(ioe);
                }
        }
}
```

شکل ۳-۲۷ کارگزار تاریخ.

جزئیات چگونگی ارتباط کارگزار با سوکت به این شرح است. کارگزار در ابتدا یک شئ PrintWriter ایجاد می‌کند که برای ارتباط با کارفرما استفاده خواهد کرد. شئ PrintWriter، این امکان را برای کارگزار فراهم می‌کند که بتواند با استفاده از متدهای خروجی معمول (print و println() در سوکت بنویسد. پردازه‌ی کارگزار با فراخوانی متد println()، تاریخ را برای کارفرما ارسال می‌کند. کارگزار، همین که تاریخ را در سوکت نوشت، سوکت کارفرما را می‌بندد و گوش‌دادن به درخواست‌های بیشتر را از سر می‌گیرد.

کارفرما با ایجاد یک سوکت و اتصال آن به درگاهی که کارگزار گوش می‌دهد با کارگزار ارتباط برقرار می‌کند. چنین کارفرمایی را با یک برنامه‌ی جاوا که در شکل ۳-۲۸ نشان داده می‌شود، پیاده‌سازی می‌کنیم. کارفرما یک Socket ایجاد کرده و اتصالی را به کارگزار با آدرس

IPای 127.0.0.1 بر روی درگاه ۶۰۱۳ درخواست می‌کند. همین که اتصال برقرار می‌شود کارفرما می‌تواند سوکت را با استفاده از دستورات معمول جریان I/O بخواند. بعد از این که تاریخ را از کارگزار دریافت کرد، کارفرما سوکت را بسته و خارج می‌شود. آدرس 127.0.0.1 یک آدرس IP خاصی است که به **loopback** معروف است. زمانی که کامپیوتری به آدرس IPای 127.0.0.1 رجوع می‌کند، در واقع به خودش رجوع می‌کند. این راهکار به یک کارفرما و کارگزار امکان می‌دهد که بتوانند با استفاده از پروتکل TCP/IP در یک میزبان با همدیگر ارتباط برقرار کنند. آدرس IPای 127.0.0.1 می‌تواند با آدرس IPای میزبان دیگری که کارگزار تاریخ را اجرا می‌کند، جایگزین شود. علاوه بر آدرس IP می‌توان از نام واقعی میزبان، نظیر www.westminstercollege.edu نیز استفاده کرد.

```
import java.net.*;
import java.io.*;
public class DateClient {
        public static void main(String[] args) {
                try {
                        /* make connection to server socket */
                        Socket sock = new Socket("127.0.0.1", 6013);

                        InputStream in = sock.getInputStream();
                        BufferedReader bin = new
                            BufferedReader(new InputStreamReader(in));

                        /* read the date from the socket */
                        String line;
                        while( (line = bin.readLine()) != null)
                           System.out.println(line);
                        /* close the socket connection */
                        sock.close();
                }
                catch (IOException ioe) {System.err.println(ioe);}
        }
}
```

شکل ۳-۲۸ کارفرمای تاریخ.

ارتباط با استفاده از سوکت‌ها—اگرچه رایج و کارآمد است—به عنوان شکل سطح پایینی از ارتباط میان پردازه‌های توزیع‌شده محسوب می‌شود. یک دلیل آن این است که سوکت‌ها تنها امکان تبادل جریان فاقد ساختاری از بایت‌ها را میان نخ‌های ارتباطی فراهم می‌کنند. لذا، مسؤولیت آن بر عهده‌ی برنامه‌ی کارفرما یا کارگزار است که بر روی داده‌ها یک ساختار اعمال کنند. در بخش بعدی، به یک متد ارتباطی سطح بالاتر می‌پردازیم که عبارت است از: فراخوانی‌های روال دور (RPC).

۳-۸-۲ فراخوانی‌های روال دور

یکی از رایج‌ترین اَشکال سرویس دور، تئوری RPC است که به‌عنوان شیوه‌ای برای تجرید راهکار فراخوانی روال جهت استفاده در میان سیستم‌هایی با اتصالات شبکه طراحی شد. این سرویس در بسیاری از وجوه شبیه راهکار IPC می‌باشد که در بخش ۳-۴ بیان شد و معمولاً در بالای چنین سیستمی ساخته می‌شود. با این وجود، در اینجا چون با محیطی سر و کار داریم که پردازه‌ها در سیستم‌های جداگانه‌ای اجرا می‌شوند، بایستی برای فراهم‌نمودن سرویس دور از یک طرح ارتباطی مبتنی بر پیغام استفاده کنیم.

در مقایسه با امکان IPC، پیغام‌های ارتباط RPC خوش‌ساخت بوده و از این رو دیگر فقط بسته‌هایی از داده‌ها نیستند. هر پیغام به یک گماشته‌ی RPC که به درگاهی در سیستم دور گوش می‌دهد روانه می‌شود و شامل شناسه‌ی مشخص‌کننده‌ی تابع اجرایی و پارامترهایی است که می‌بایست به تابع ارسال شوند. سپس، تابع مورد نظر مطابق درخواست اجرا شده و هرگونه خروجی که داشته باشد، دوباره با یک پیغام جداگانه‌ای به درخواست‌دهنده فرستاده می‌شوند.

پورت در این متن، به‌سادگی شماره‌ای است که در ابتدای بسته‌ی پیغام گذاشته می‌شود. در حالی که، سیستم در اصل فقط یک آدرس شبکه دارد، با همان آدرس می‌تواند درگاه‌های زیادی فراهم کند تا سرویس‌های متعدد شبکه را که پشتیبانی می‌کند از هم تمییز بدهد. اگر پردازه‌ای از دور سرویس لازم داشته باشد، پیغام را به مقصد درگاه مناسبی روانه می‌سازد. برای نمونه، اگر سیستمی بخواهد به پردازه‌های دیگر این اجازه را بدهد که بتوانند لیست تمامی کاربران فعلی آن را بگیرند، برای پشتیبانی این نوع RPC، گماشته‌ای متصل به یک درگاه—بر فرض، درگاه ۳۰۲۷—خواهد گذاشت. هر سیستم دور می‌تواند اطلاعات مورد نیاز خود (یعنی، لیست کاربران فعلی) را با ارسال یک پیغام RPC به درگاه ۳۰۲۷ در کارگزار بدست آورد. داده‌ها در قالب یک پیغام پاسخ، دریافت خواهند شد.

معانی[1] RPC به یک کارفرما امکان می‌دهد تا یک روال میزبان دور را چنان احضار کند که یک روال محلی را احضار خواهد کرد. سیستم RPC با پنهان‌سازی جزئیات، انجام ارتباط را با گذاشتن **ته‌کدی**[2] در طرف کارفرما فراهم می‌کند. نوعاً، ته‌کد مجزایی برای هر یک از روال‌های دور مجزا وجود دارد. زمانی که کارفرما یک روال دور را احضار می‌کند، سیستم RPC، ته‌کد مناسبی را با عبور پارامترهای مناسب روال دور فرا می‌خواند. این ته‌کد، درگاه روی کارگزار را یافته و پارامترها را **می‌چیند**[3]. سپس، ته‌کد با استفاده از عبور پیغام، یک پیغام به کارگزار می‌فرستد. ته‌کد مشابهی در طرف کارگزار این پیغام را دریافت کرده و روال را در کارگزار احضار می‌کند. اگر لازم باشد، مقادیر برگشتی دومرتبه با استفاده از تکنیک یکسانی به کارفرما برگردانده می‌شوند. در سیستم‌های ویندوز، کد ته‌کد از یک توصیف نوشته شده به **زبان تعریف رابط مایکروسافت** (**MIDL**) کامپایل می‌شود که برای تعریف رابط‌های میان برنامه‌های کارفرما و کارگزار استفاده می‌شود.

1. Semantics
2. Stub
3. Marshals

چینش پارامتر، مورد نگران‌کننده‌ی تفاوت‌ها در نمایش داده‌ها بر روی ماشین‌های کارفرما و کارگزار را مد نظر قرار می‌دهد. نمایش ۳۲ بیتی اعداد صحیح را در نظر بگیرید. برخی سیستم‌ها (که به **بزرگ بالانشین**[1] معروفند) از آدرس بالای حافظه برای ذخیره‌ی بایت با ارزش استفاده می‌کنند، حال آنکه سیستم‌های دیگری هستند (که به **کوچک بالانشین**[2] معروفند) که بایت کم ارزش را در آدرس بالای حافظه ذخیره می‌کنند. هیچ یک از این ترتیب‌ها به خودی خود «بهتر» نیستند؛ بلکه، انتخاب به یک معماری کامپیوتر مربوط می‌شود. بسیاری از سیستم‌های RPC برای حل این قبیل مشکلات، یک نمایش مستقل از ماشین را برای داده‌ها تعریف می‌کنند. یکی از این نمایش‌ها به **نمایش بیرونی داده‌ها**[3] (**XDR**) معروف است. در طرف کارفرما، موقع چیدن پارامترها، داده‌های وابسته به ماشین، پیش از ارسال آنها به کارگزار، به XDR تبدیل می‌شوند. در طرف کارگزار، داده‌های XDR تفکیک شده و به شکل نمایش مربوط به ماشین کارگزار تبدیل می‌شوند.

مورد مهم دیگری که وجود دارد مربوط به معانی یک فراخوانی است. با این که فراخوانی‌های روال محلی در شرایط بسیار اندکی دچار مشکل می‌شوند، RPCها در نتیجه‌ی خطاهای مرسوم شبکه‌ها ممکن است با خطا مواجه شده، یا تکثیر شده و بیش از یکبار اجرا شوند. یک راه چاره برای این مسأله آن است که سیستم‌عامل تضمین نماید پیغام‌ها *دقیقاً یکبار* به جای *حداکثر یکبار* مورد عمل واقع می‌شوند. اکثر فراخوانی‌های روال محلی، کارکرد «دقیقاً یکبار» را دارند، اما پیاده‌سازی آن دشوارتر است.

نخست، «حداکثر یکبار» را در نظر بگیرید. این معنی با الحاق یک زمان‌مهر[4] به هر پیغام قابل حصول است. کارگزار باید سابقه‌ای از تمامی زمان‌مهر پیغام‌هایی که قبلاً پردازش کرده است، نگه‌داری کند یا سابقه را چنان بزرگ بگیرد تا مطمئن شود که پیغام‌های تکراری قابل تشخیص خواهند بود. پیغام‌های ورودی که زمان‌مهر آنها از قبل در سابقه موجود است، چشم‌پوشی می‌شوند. در این صورت، کارفرما می‌تواند یک پیغام را یکبار یا بیشتر ارسال کند، اما مطمئن است که تنها یکبار اجرا می‌شوند.

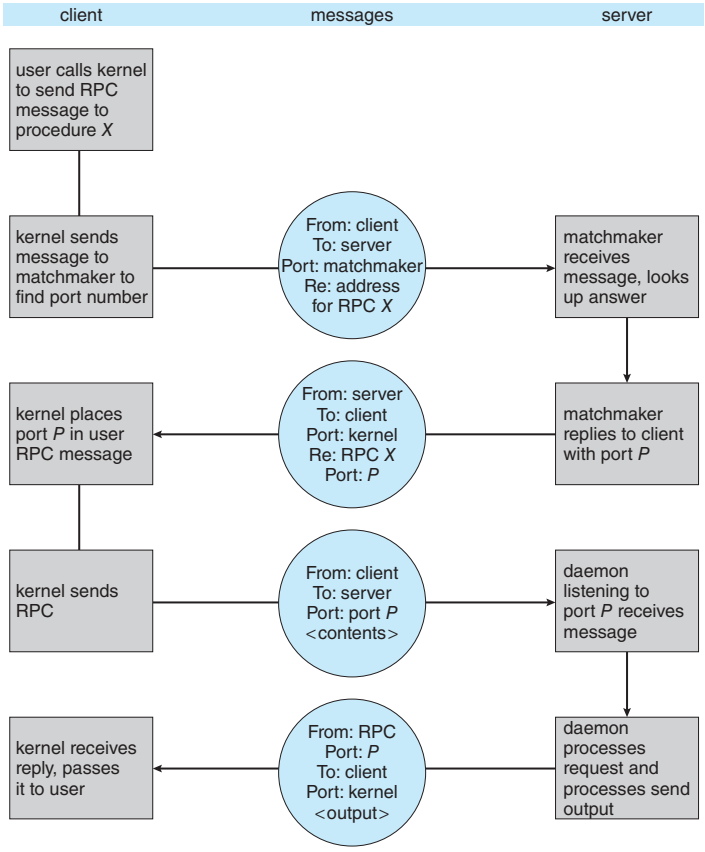
برای «دقیقاً یکبار» لازم داریم این خطر را که کارگزار هرگز درخواست را دریافت نخواهد کرد، حذف کنیم. برای انجام این کار، کارگزار بایستی پروتکل «حداکثر یکبار» را که در بالا مطرح شد پیاده‌سازی کند، اما می‌بایست یک تأییدیه هم مبنی بر دریافت و اجرای فراخوانی RPC به کارفرما بفرستد. این پیغام‌های ACK در کل شبکه رایج هستند. کارفرما بایستی هر فراخوانی RPC را با تناوبی مجدداً ارسال کند تا این که ACK آن فراخوانی را دریافت نماید.

باز مورد مهم دیگری که نگران‌کننده است به ارتباط میان کارگزار و کارفرما بر می‌گردد. در فراخوانی معمول روال‌ها، شکلی از پیوند در طی زمان اتصال، بارگذاری یا اجرا اتفاق می‌افتد (فصل ۹) تا این که نام روال فراخوانی با آدرس حافظه‌ی روال فراخوانی‌شده جایگزین شود. طرح RPC نیازمند پیوند مشابهی از درگاه کارفرما و کارگزار می‌باشد، اما کارفرما چگونه شماره‌ی

1. Big-endian
2. Little-endian
3. External data representation
4. Timestamp

درگاه‌های کارگزار را بدست می‌آورد؟ هیچ یک از سیستم‌ها اطلاعات کاملی از یکدیگر ندارند، چون حافظه‌ی مشترکی ندارند.

دو رویکرد رایج است. اولاً، اطلاعات پیوند می‌تواند پیشاپیش به شکل آدرس‌های درگاه ثابت تعیین شود. در زمان کامپایل، هر فراخوانی RPC، یک شماره درگاه ثابت منتسب به آن دارد. همین که برنامه کامپایل می‌شود، کارگزار نمی‌تواند شماره درگاه سرویس درخواستی را تغییر بدهد. ثانیاً، پیوند می‌تواند به صورت پویا با یک راهکار ملاقات انجام شود. سیستم‌عامل، نوعاً گماشته‌ای را برای ملاقات (که **دلال**[1] نیز نامیده می‌شود) بر روی یک درگاه ثابت RPC فراهم می‌کند. سپس، کارفرما پیغامی حاوی نام RPC به گماشته‌ی ملاقات می‌فرستد و آدرس درگاه RPC را که می‌خواهد اجرا کند، درخواست می‌نماید. شماره‌ی درگاه برگردانده می‌شود و مادامی که پردازه خاتمه نیافته (یا کارگزار از کار نیافته)، فراخوانی‌های RPC می‌توانند به آن درگاه ارسال شوند. این روش، به دلیل درخواست آغازین، سربار اضافه‌تری دارد اما انعطاف آن از روش اول بیشتر است. شکل ۳–۲۹، یک تعامل نمونه را نشان می‌دهد.



شکل ۳–۲۹ اجرای فراخوانی روال دور (RPC).

1. Matchmaker

طرح RPC برای پیاده‌سازی سیستم‌فایل توزیع‌شده (فصل ۱۹) مفید است. چنین سیستمی می‌تواند به صورت مجموعه‌ای از گماشتگان RPC و کارفرماها پیاده‌سازی شود. پیغام‌ها به مقصد درگاه سیستم‌فایل توزیع‌شده بر روی کارگزار روانه می‌شوند که قرار است عملیات فایل در آنجا انجام شود. پیغام، عملیات دیسک مورد نظر را در بر دارد. عملیات دیسک می‌تواند read()، write()، rename()، delete()، یا status() باشد که متناظر فراخوانی‌های سیستمی معمول مربوط به فایل هستند. پیغام برگشتی، هرگونه داده‌ای را که از فراخوانی حاصل می‌شود شامل است که توسط گماشته‌ی DFS از جانب کارفرما اجرا می‌شوند. برای نمونه، یک پیغام می‌تواند درخواست انتقال کل یک فایل برای کارفرما باشد یا محدود به درخواست ساده‌ی یک بلاک باشد. در مورد دوم، اگر قرار است که کل فایل منتقل شود، چندین درخواست ممکن است لازم شود.

۳-۸-۲-۱ RPC اندروید

هرچند که RPCها نوعاً با محاسبات کارفرما-کارگزار در یک سیستم توزیع‌شده مرتبط هستند، می‌توانند به‌صورت شکلی از IPC در میان پردازه‌هایی که روی سیستم یکسانی اجرا دارند استفاده شوند. سیستم‌عامل اندروید مجموعه‌ای غنی از راهکارهای IPC دارد که در چارچوب **پیونددهنده‌ی**[1] آن قرار دارند، اعم از RPCهایی که به یک پردازه اجازه می‌دهند تا سرویس‌هایی را از پردازه‌ی دیگر درخواست کند.

اندروید **مؤلفه‌ی** یک **برنامه** را به‌عنوان یک بلاک سازنده‌ی اساسی تعریف می‌کند که برنامه‌ی سودمندی برای یک برنامه‌ی اندروید فراهم کرده و یک app می‌تواند چندین مؤلفه‌ی برنامه را ترکیب کند تا کارکردی برای یک app فراهم شود. یکی از مؤلفه‌های این چنینی، **سرویس** است که هیچ رابط کاربری ندارد اما در عوض، در حالی که عملیات با زمان اجرای طولانی را اجرا می‌کند یا کاری برای پردازه‌های دور انجام می‌دهد در پس‌زمینه اجرا می‌شود. مثال‌هایی از سرویس‌ها عبارتند از اجرای موسیقی در پس‌زمینه و بازیابی داده‌ها از یک اتصال شبکه از طرف پردازه‌ای دیگر، با این کار پردازه‌ی دیگر، مادامی که داده‌ها دانلود می‌شوند مسدود نمی‌شود. موقعی که یک app کارفرما متد bindService() یک سرویس را احضار می‌کند، آن سرویس «پیوند می‌خورد» و در دسترس قرار می‌گیرد تا ارتباط کارفرما-کارگزار را با استفاده از عبور پیغام و یا RPC فراهم کند.

یک سرویس پیوندخورده بایستی کلاس Service اندروید را گسترش بدهد و می‌بایست متد onBind() را پیاده‌سازی کند تا موقعی که یک کارفرما bindService() را فراخوانی می‌کند احضار شود. در حالت عبور پیغام، متد onBind() یک سرویس Messenger بر می‌گرداند که برای ارسال پیغام‌ها از کارفرما به سرویس استفاده می‌شود. سرویس Messenger صرفاً یک طرفه است؛ اگر سرویس می‌بایست یک پاسخ به کارفرما ارسال کند، بایستی کارفرما نیز یک سرویس Messenger فراهم کند که در فیلد replyTo شئ Message ارسالی به سرویس لحاظ می‌شود. در ادامه، سرویس می‌تواند پیغام‌هایی را دوباره به کارفرما ارسال کند.

1. Binder

متد ()doBind به‌منظور تدارک RPCها بایستی یک رابطی برگرداند که معرف متدهایی در شی دور است که کارفرماها برای تعامل با سرویس استفاده می‌کنند. این رابط با نحو معمول جاوا نوشته می‌شود و از زبان تعریف رابط اندروید[1] (AIDL) استفاده می‌کند تا ته‌فایل‌ها[2] را ایجاد کند که به‌عنوان رابط کارفرما برای سرویس‌های دور می‌مانند.

ما در اینجا، به‌اختصار، فرایند لازم برای فراهم‌ساختن یک سرویس دور عمومی به‌نام ()remoteMethod را با استفاده از AIDL و سرویس پیونددهنده مطرح می‌کنیم. رابط سرویس دور به‌صورت زیر است:

```
/* RemoteService.aidl */
interface RemoteService
{
        boolean remoteMethod(int x, double y);
}
```

این فایل به‌صورت RemoteService.aidl نوشته می‌شود. ابزار توسعه‌ی اندروید از آن استفاده خواهد کرد تا یک رابط java. از فایل aidl. تولید کند، همین‌طور یک ته‌کدی که به‌عنوان رابط RPC برای این سرویس عمل می‌کند. کارگزار بایستی رابط تولید شده توسط فایل aidl. را پیاده‌سازی کند و پیاده‌سازی این رابط در زمانی که کارفرما ()remoteMethod را احضار می‌کند فراخوانی خواهد شد.

موقعی که یک کارفرما ()bindService را فراخوانی می‌کند، متد ()onBind روی کارگزار احضار می‌شود و ته‌کد شئ RemoteService را به کارفرما بر می‌گرداند. در ادامه، کارفرما می‌تواند مند دور را به‌صورت زیر احضار کند:

```
RemoteService service;
        . . .
service.remoteMethod(3, 0.14);
```

در داخل، چارچوب پیونددهنده‌ی اندروید به چینش پارامترها رسیدگی می‌کند، پارامترهای چیده شده را میان پردازه‌ها انتقال می‌دهد، و پیاده‌سازی لازم سرویس را احضار می‌کند، همین‌طور، هرگونه مقدار برگشتی را دوباره به پردازه‌ی کارفرما ارسال می‌کند.

۳-۹ خلاصه

- پردازه یک برنامه‌ی در حال اجراست و وضعیت فعالیت جاری یک پردازه با شمارنده‌ی برنامه، در کنار سایر ثبات‌ها مشخص می‌شود.
- چیدمان یک پردازه در درون حافظه با چهار بخش نمایش داده می‌شود: (۱) متن، (۲) داده‌ها، (۳) هیپ، و (۴) پشته.
- همچنان‌که یک پردازه اجرا می‌شود، تغییر حالت می‌دهد. چهار حالت کلی برای یک

1. Android Interface Definition Language (AIDL)
2. Stub files